

# Assignment 3: Time-Series Forecasting

Curtis Meister

March 30, 2026

```
library(tidyverse)
library(keras)
library(tensorflow)
library(reticulate)
```

```
set.seed(123)
tf$random$set_seed(as.integer(123))
```

```
zip_path <- get_file(
  fname = "jena_climate_2009_2016.csv.zip",
  origin = "https://storage.googleapis.com/tensorflow/tf-keras-datasets/jena_climate_2009_2016",
  extract = TRUE
)

csv_path <- file.path(dirname(zip_path), "jena_climate_2009_2016.csv")
weather_data <- read.csv(csv_path)

glimpse(weather_data)
```

```
## Rows: 420,551
## Columns: 15
## $ Date.Time      <chr> "01.01.2009 00:10:00", "01.01.2009 00:20:00", "01.01.2~
## $ p..mbar.       <dbl> 996.52, 996.57, 996.53, 996.51, 996.51, 996.50, 996.50~
## $ T..degC.       <dbl> -8.02, -8.41, -8.51, -8.31, -8.27, -8.05, -7.62, -7.62~
## $ Tpot..K.       <dbl> 265.40, 265.01, 264.91, 265.12, 265.15, 265.38, 265.81~
## $ Tdew..degC.    <dbl> -8.90, -9.28, -9.31, -9.07, -9.04, -8.78, -8.30, -8.36~
## $ rh....        <dbl> 93.3, 93.4, 93.9, 94.2, 94.1, 94.4, 94.8, 94.4, 93.8, ~
## $ VPmax..mbar.   <dbl> 3.33, 3.23, 3.21, 3.26, 3.27, 3.33, 3.44, 3.44, 3.36, ~
## $ VPact..mbar.   <dbl> 3.11, 3.02, 3.01, 3.07, 3.08, 3.14, 3.26, 3.25, 3.15, ~
## $ VPdef..mbar.   <dbl> 0.22, 0.21, 0.20, 0.19, 0.19, 0.19, 0.18, 0.19, 0.21, ~
## $ sh..g.kg.      <dbl> 1.94, 1.89, 1.88, 1.92, 1.92, 1.96, 2.04, 2.03, 1.97, ~
## $ H2OC..mmol.mol.<dbl> 3.12, 3.03, 3.02, 3.08, 3.09, 3.15, 3.27, 3.26, 3.16, ~
## $ rho..g.m..3.   <dbl> 1307.75, 1309.80, 1310.24, 1309.19, 1309.00, 1307.86, ~
## $ ww..m.s.       <dbl> 1.03, 0.72, 0.19, 0.34, 0.32, 0.21, 0.18, 0.19, 0.28, ~
## $ max..ww..m.s.  <dbl> 1.75, 1.50, 0.63, 0.50, 0.63, 0.63, 0.63, 0.50, 0.75, ~
## $ wd..deg.       <dbl> 152.3, 136.1, 171.6, 198.0, 214.3, 192.7, 166.5, 118.6~
```

```
dim(weather_data)
```

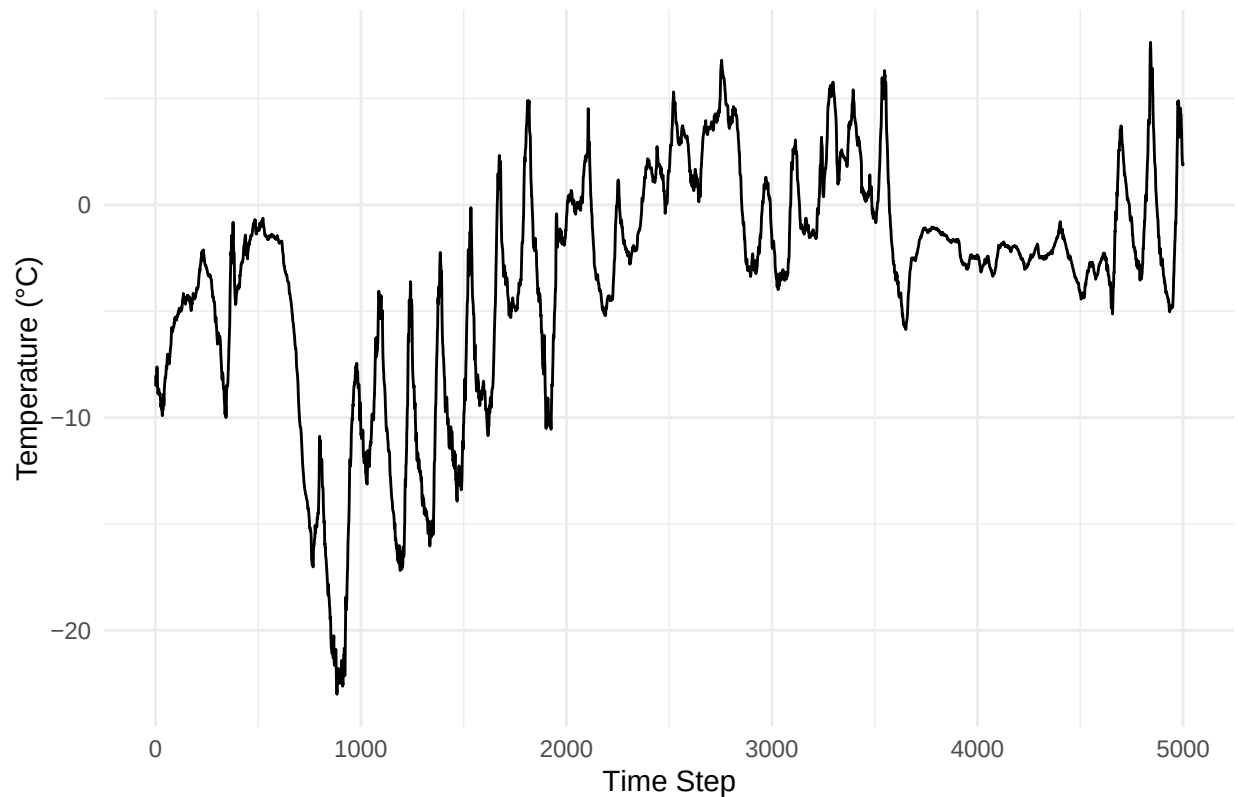
```
## [1] 420551      15
```

```
head(weather_data)
```

```
##           Date.Time p..mbar. T..degC. Tpot..K. Tdew..degC. rh....
## 1 01.01.2009 00:10:00  996.52   -8.02   265.40       -8.90   93.3
## 2 01.01.2009 00:20:00  996.57   -8.41   265.01       -9.28   93.4
## 3 01.01.2009 00:30:00  996.53   -8.51   264.91       -9.31   93.9
## 4 01.01.2009 00:40:00  996.51   -8.31   265.12       -9.07   94.2
## 5 01.01.2009 00:50:00  996.51   -8.27   265.15       -9.04   94.1
## 6 01.01.2009 01:00:00  996.50   -8.05   265.38       -8.78   94.4
##   VPmax..mbar. VPact..mbar. VPdef..mbar. sh..g.kg. H2OC..mmol.mol. rho..g.m..3.
## 1         3.33         3.11         0.22      1.94              3.12      1307.75
## 2         3.23         3.02         0.21      1.89              3.03      1309.80
## 3         3.21         3.01         0.20      1.88              3.02      1310.24
## 4         3.26         3.07         0.19      1.92              3.08      1309.19
## 5         3.27         3.08         0.19      1.92              3.09      1309.00
## 6         3.33         3.14         0.19      1.96              3.15      1307.86
##   wv..m.s. max..wv..m.s. wd..deg.
## 1      1.03          1.75     152.3
## 2      0.72          1.50     136.1
## 3      0.19          0.63     171.6
## 4      0.34          0.50     198.0
## 5      0.32          0.63     214.3
## 6      0.21          0.63     192.7
```

```
ggplot(weather_data[1:5000, ], aes(x = 1:5000, y = T..degC.)) +
  geom_line() +
  theme_minimal() +
  labs(
    title = "Temperature Over Time (First 5,000 Observations)",
    x = "Time Step",
    y = "Temperature (°C)"
  )
```

Temperature Over Time (First 5,000 Observations)



```
sampling_rate <- as.integer(6)
sequence_length <- as.integer(120)
delay <- as.integer(144)
batch_size <- as.integer(256)
```

```
n <- nrow(weather_data)
```

```
train_end <- as.integer(round(0.5 * n))
val_end <- as.integer(round(0.75 * n))
```

```
train_data <- weather_data[1:train_end, ]
val_data <- weather_data[(train_end + 1):val_end, ]
test_data <- weather_data[(val_end + 1):n, ]
```

```
train_matrix <- as.matrix(train_data[, -1])
val_matrix <- as.matrix(val_data[, -1])
test_matrix <- as.matrix(test_data[, -1])
```

```
train_mean <- colMeans(train_matrix)
train_sd <- apply(train_matrix, 2, sd)
```

```
train_matrix <- scale(train_matrix, center = train_mean, scale = train_sd)
```

```

val_matrix    <- scale(val_matrix, center = train_mean, scale = train_sd)
test_matrix   <- scale(test_matrix, center = train_mean, scale = train_sd)

target_col_index <- which(colnames(weather_data[, -1]) == "T..degC.")
target_col_index <- as.integer(target_col_index)

```

```

make_dataset <- function(data_matrix, targets, sequence_length, delay,
                          sampling_rate = 6L, batch_size = 256L, shuffle = FALSE) {
  timeseries_dataset_from_array(
    data = data_matrix,
    targets = targets[(sequence_length + delay):length(targets)],
    sequence_length = as.integer(sequence_length),
    sampling_rate = as.integer(sampling_rate),
    batch_size = as.integer(batch_size),
    shuffle = shuffle,
    start_index = as.integer(0),
    end_index = as.integer(nrow(data_matrix) - delay - 1)
  )
}

```

```

train_ds <- make_dataset(
  data_matrix = train_matrix,
  targets = train_data$T..degC.,
  sequence_length = sequence_length,
  delay = delay,
  sampling_rate = sampling_rate,
  batch_size = batch_size,
  shuffle = TRUE
)

```

```

val_ds <- make_dataset(
  data_matrix = val_matrix,
  targets = val_data$T..degC.,
  sequence_length = sequence_length,
  delay = delay,
  sampling_rate = sampling_rate,
  batch_size = batch_size,
  shuffle = FALSE
)

```

```

test_ds <- make_dataset(
  data_matrix = test_matrix,
  targets = test_data$T..degC.,
  sequence_length = sequence_length,
  delay = delay,
  sampling_rate = sampling_rate,
  batch_size = batch_size,

```

```

    shuffle = FALSE
  )

evaluate_naive <- function(dataset) {
  maes <- c()
  it <- reticulate::as_iterator(dataset)

  repeat {
    batch <- tryCatch(
      reticulate::iter_next(it),
      error = function(e) NULL
    )
    if (is.null(batch)) break
    inputs <- batch[[1]]
    targets <- batch[[2]]
    preds <- as.numeric(inputs[, dim(inputs)[2], target_col_index])
    actual <- as.numeric(targets)
    maes <- c(maes, mean(abs(preds - actual)))
  }
  mean(maes)
}

naive_mae <- evaluate_naive(val_ds)
naive_mae

```

```
## [1] 10.3439
```

```

fit_model <- function(model, train_ds, val_ds, epochs = 10) {
  history <- model %>% fit(
    train_ds,
    epochs = as.integer(epochs),
    validation_data = val_ds,
    verbose = 0
  )

  metric_name <- if ("val_mae" %in% names(history$metrics)) {
    "val_mae"
  } else if ("val_mean_absolute_error" %in% names(history$metrics)) {
    "val_mean_absolute_error"
  } else {
    stop("Validation MAE metric not found in history$metrics")
  }

  tail(history$metrics[[metric_name]], 1)
}

```

```
dense_model <- keras_model_sequential() %>%
  layer_flatten(input_shape = c(sequence_length, ncol(train_matrix))) %>%
  layer_dense(units = 32, activation = "relu") %>%
  layer_dropout(rate = 0.3) %>%
  layer_dense(units = 1)

dense_model %>% compile(
  optimizer = optimizer_rmsprop(),
  loss = "mse",
  metrics = list("mae")
)

dense_mae <- fit_model(dense_model, train_ds, val_ds)
```

```
dense_mae
```

```
## [1] 0.7920747
```

```
lstm_model <- keras_model_sequential() %>%
  layer_lstm(
    units = 32,
    recurrent_dropout = 0.25,
    input_shape = c(sequence_length, ncol(train_matrix))
  ) %>%
  layer_dense(units = 1)

lstm_model %>% compile(
  optimizer = optimizer_rmsprop(),
  loss = "mse",
  metrics = list("mae")
)

lstm_mae <- fit_model(lstm_model, train_ds, val_ds)
```

```
lstm_mae
```

```
## [1] 1.055111
```

```
gru_model <- keras_model_sequential() %>%
  layer_gru(
    units = 32,
    recurrent_dropout = 0.25,
    input_shape = c(sequence_length, ncol(train_matrix))
  ) %>%
  layer_dense(units = 1)
```

```
gru_model %>% compile(
  optimizer = optimizer_rmsprop(),
  loss = "mse",
  metrics = list("mae")
)

gru_mae <- fit_model(gru_model, train_ds, val_ds)
```

```
gru_mae
```

```
## [1] 0.8321033
```

```
stacked_gru <- keras_model_sequential() %>%
  layer_gru(
    units = 32,
    return_sequences = TRUE,
    recurrent_dropout = 0.25,
    input_shape = c(sequence_length, ncol(train_matrix))
  ) %>%
  layer_gru(
    units = 32,
    recurrent_dropout = 0.25
  ) %>%
  layer_dense(units = 1)

stacked_gru %>% compile(
  optimizer = optimizer_rmsprop(),
  loss = "mse",
  metrics = list("mae")
)

stacked_mae <- fit_model(stacked_gru, train_ds, val_ds)
```

```
stacked_mae
```

```
## [1] 0.601744
```

```
conv_gru <- keras_model_sequential() %>%
  layer_conv_1d(
    filters = 32,
    kernel_size = 5,
    activation = "relu",
    input_shape = c(sequence_length, ncol(train_matrix))
  ) %>%
  layer_max_pooling_1d(pool_size = 3) %>%
```

```

layer_gru(units = 32) %>%
layer_dense(units = 1)

conv_gru %>% compile(
  optimizer = optimizer_rmsprop(),
  loss = "mse",
  metrics = list("mae")
)

conv_mae <- fit_model(conv_gru, train_ds, val_ds)

```

```
conv_mae
```

```
## [1] 0.5960037
```

Now I am going to test GRU and Conv1D+GRU across unit sizes of 16, 32, 64, and 128. This will help determine how number of units effects performance.

```

unit_sizes <- c(16, 32, 64, 128)

gru_unit_results <- map_dfr(unit_sizes, function(units) {
  model <- keras_model_sequential() %>%
    layer_gru(
      units = units,
      recurrent_dropout = 0.25,
      input_shape = c(sequence_length, ncol(train_matrix))
    ) %>%
    layer_dense(units = 1)

  model %>% compile(
    optimizer = optimizer_rmsprop(),
    loss = "mse",
    metrics = list("mae")
  )

  val_mae <- fit_model(model, train_ds, val_ds, epochs = 5)
  tibble(Model = paste0("GRU (", units, " units)", MAE = val_mae)
})

```

```
gru_unit_results
```

```

## # A tibble: 4 x 2
##   Model      MAE
##   <chr>      <dbl>
## 1 GRU (16 units) 1.97
## 2 GRU (32 units) 1.48

```

```
## 3 GRU (64 units) 0.917
## 4 GRU (128 units) 1.03
```

```
conv_gru_unit_results <- map_dfr(unit_sizes, function(units) {
  model <- keras_model_sequential() %>%
    layer_conv_1d(
      filters = units,
      kernel_size = 5,
      activation = "relu",
      input_shape = c(sequence_length, ncol(train_matrix))
    ) %>%
    layer_max_pooling_1d(pool_size = 3) %>%
    layer_gru(units = units) %>%
    layer_dense(units = 1)

  model %>% compile(
    optimizer = optimizer_rmsprop(),
    loss = "mse",
    metrics = list("mae")
  )

  val_mae <- fit_model(model, train_ds, val_ds, epochs = 5)
  tibble(Model = paste0("Conv1D+GRU (", units, " units)"), MAE = val_mae)
})
```

```
conv_gru_unit_results
```

```
## # A tibble: 4 x 2
##   Model          MAE
##   <chr>          <dbl>
## 1 Conv1D+GRU (16 units) 1.19
## 2 Conv1D+GRU (32 units) 1.000
## 3 Conv1D+GRU (64 units) 0.640
## 4 Conv1D+GRU (128 units) 0.405
```

## Summary of All Results

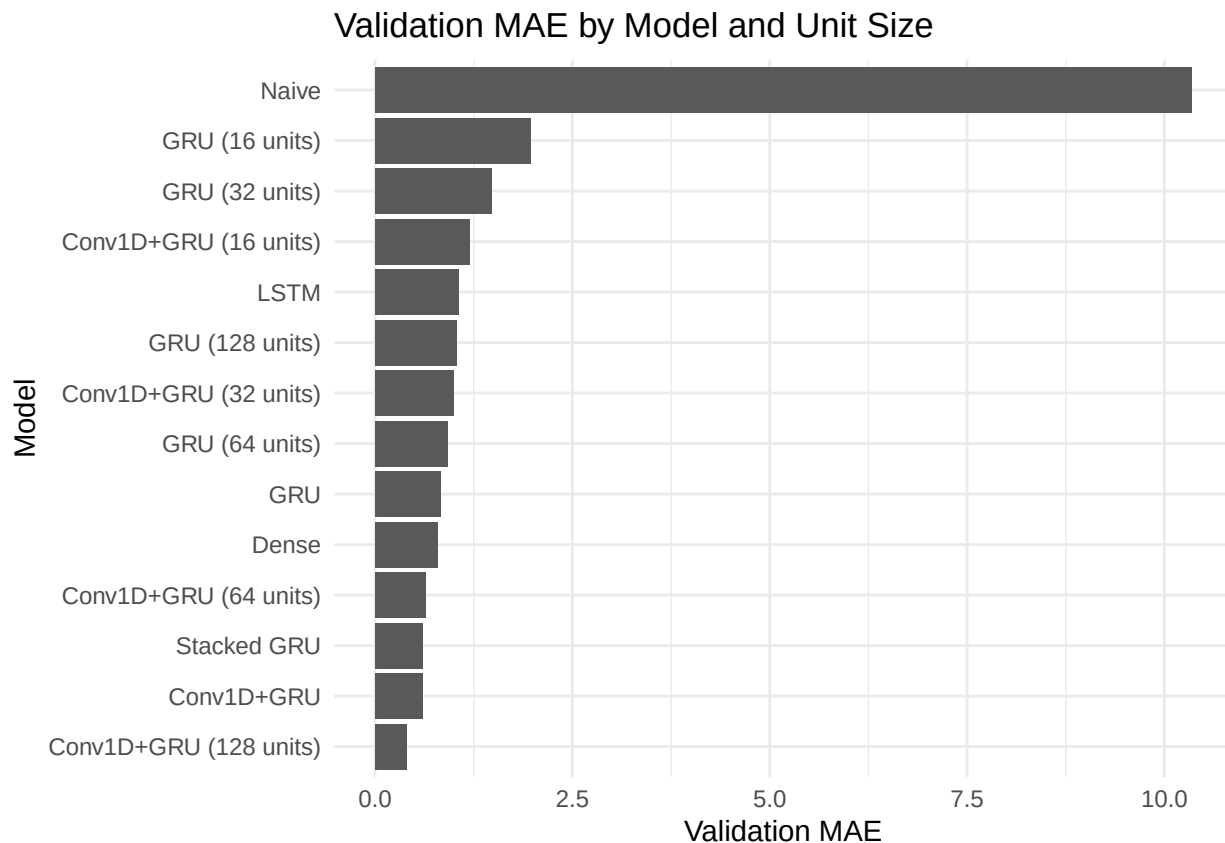
```
results <- bind_rows(
  tibble(
    Model = c("Naive", "Dense", "LSTM", "GRU", "Stacked GRU", "Conv1D+GRU"),
    MAE = c(naive_mae, dense_mae, lstm_mae, gru_mae, stacked_mae, conv_mae)
  ),
  gru_unit_results,
  conv_gru_unit_results
) %>%
```

```
arrange(MAE)
```

```
results
```

```
## # A tibble: 14 x 2
##   Model      MAE
##   <chr>    <dbl>
## 1 Conv1D+GRU (128 units) 0.405
## 2 Conv1D+GRU           0.596
## 3 Stacked GRU          0.602
## 4 Conv1D+GRU (64 units) 0.640
## 5 Dense                0.792
## 6 GRU                  0.832
## 7 GRU (64 units)       0.917
## 8 Conv1D+GRU (32 units) 1.000
## 9 GRU (128 units)      1.03
## 10 LSTM                1.06
## 11 Conv1D+GRU (16 units) 1.19
## 12 GRU (32 units)       1.48
## 13 GRU (16 units)       1.97
## 14 Naive                10.3
```

```
results %>%
  ggplot(aes(x = reorder(Model, MAE), y = MAE)) +
  geom_col() +
  coord_flip() +
  theme_minimal() +
  labs(
    title = "Validation MAE by Model and Unit Size",
    x = "Model",
    y = "Validation MAE"
  )
```



```
best_model_name <- results$Model[1]
cat("Best model:", best_model_name, "\n")
```

```
## Best model: Conv1D+GRU (128 units)
```

```
get_units <- function(name) {
  m <- regmatches(name, regexpr("[0-9]+(=? units)", name, perl = TRUE))
  if (length(m) == 1) as.integer(m) else 32L
}

best_model <- if (best_model_name == "Dense") {
  dense_model
} else if (best_model_name == "LSTM") {
  lstm_model
} else if (best_model_name == "GRU") {
  gru_model
} else if (best_model_name == "Stacked GRU") {
  stacked_gru
} else if (best_model_name == "Conv1D+GRU") {
  conv_gru
} else if (grepl("^GRU \\(", best_model_name)) {
  u <- get_units(best_model_name)
```

```

m <- keras_model_sequential() %>%
  layer_gru(units = u, recurrent_dropout = 0.25,
            input_shape = c(sequence_length, ncol(train_matrix))) %>%
  layer_dense(units = 1)
m %>% compile(optimizer = optimizer_rmsprop(), loss = "mse", metrics = list("mae"))
fit_model(m, train_ds, val_ds)
m
} else if (grepl("^Conv1D\\++GRU \\\(", best_model_name)) {
  u <- get_units(best_model_name)
  m <- keras_model_sequential() %>%
    layer_conv_1d(filters = u, kernel_size = 5, activation = "relu",
                  input_shape = c(sequence_length, ncol(train_matrix))) %>%
    layer_max_pooling_1d(pool_size = 3) %>%
    layer_gru(units = u) %>%
    layer_dense(units = 1)
  m %>% compile(optimizer = optimizer_rmsprop(), loss = "mse", metrics = list("mae"))
  fit_model(m, train_ds, val_ds)
  m
} else {
  NULL
}

if (!is.null(best_model)) {
  cat("\nTest set evaluation for:", best_model_name, "\n")
  test_metrics <- best_model %>% evaluate(test_ds, verbose = 0)
  print(test_metrics)
} else {
  cat("Naive baseline was best on validation; no neural network test evaluation run.\n")
}

```

```

##
## Test set evaluation for: Conv1D+GRU (128 units)
##      loss      mae
## 0.1945383 0.3259089

```